Persistent Sandbox AI Master Plan

Living NPCs, durable memory, visual verification, and an auto-research loop that does not reset every run.

North Star: Build a sandbox where NPCs carry out daily activities, remember what happened, resume after restart, and give Hermes/Codex enough evidence to improve the system safely.

Document	Persistent Sandbox AI Master Plan
Primary stack	Godot + SQLite first; PostgreSQL/pgvector later if multiplayer or scale requires it.
Target	Hermes/Godot development loop: NPC routines, event logging, persistence, reflection, verification, and later model training.
Version	v1.0 - 2026-04-26

Table of Contents

1. Executive Summary
2. What Current Advanced Game AI Suggests
3. System Architecture
4. Persistence Model: What Must Never Reset
5. Database Plan: SQL First, Vector Memory Later
6. NPC Brain: Daily Life, Goals, Memory, Relationships
7. Event Logging and Evidence
8. Hermes Auto-Research Improvement Loop
9. Visual Verification With Pictures
10. Training and Neural Evolution: When It Actually Makes Sense
11. Implementation Roadmap
12. Codex Build Prompts
13. Risks, Limits, and Guardrails
14. References

1. Executive Summary

The goal is not to start by training a neural model. The first goal is to make the world durable. NPCs should wake up, work, eat, talk, fight, remember, sleep, and resume later from saved state. The AI framework should record what happened, score whether behavior improved, and only keep changes that pass tests and evidence checks.

The reliable first architecture is a classic game simulation backed by a database. Godot runs the fast behavior. SQLite stores the world truth. Hermes/Codex performs slower reasoning, research, reflection, code changes, and verification.

Best first milestone: An NPC remembers that the player stole from them, the game closes, the game reopens, and the NPC behaves differently because the memory and relationship state persisted.

The plan has three layers

- Persistent world: location, time, objects, inventory, damage, quests, schedules, and NPC state survive restarts.
- Persistent NPC memory: NPCs form memories, reflections, beliefs, and relationship scores from logged events.
- Learning framework: Hermes reviews logs, proposes changes, runs tests, captures screenshots, verifies outcomes, and stores lessons.

Core recommendation

Decision	Recommendation
Database	Use SQLite first. Move to PostgreSQL only when multiplayer, remote sync, or high scale appears.
Memory	Use SQL for exact facts. Add vector search later for fuzzy memory retrieval.
Training	Do not train first. Collect clean state-action-outcome logs first.
AI control	Use LLMs for high-level planning, reflection, dialogue, and research - not per-frame movement.
Verification	Use tests, JSON logs, scorecards, screenshots, and vision checks.

2. What Current Advanced Game AI Suggests

Current game-AI work points toward persistent characters, memory, reflection, autonomy, and real-time dialogue. The important lesson is not that every NPC needs a giant model. The lesson is that believable behavior comes from combining state, memory, planning, perception, and guardrails.

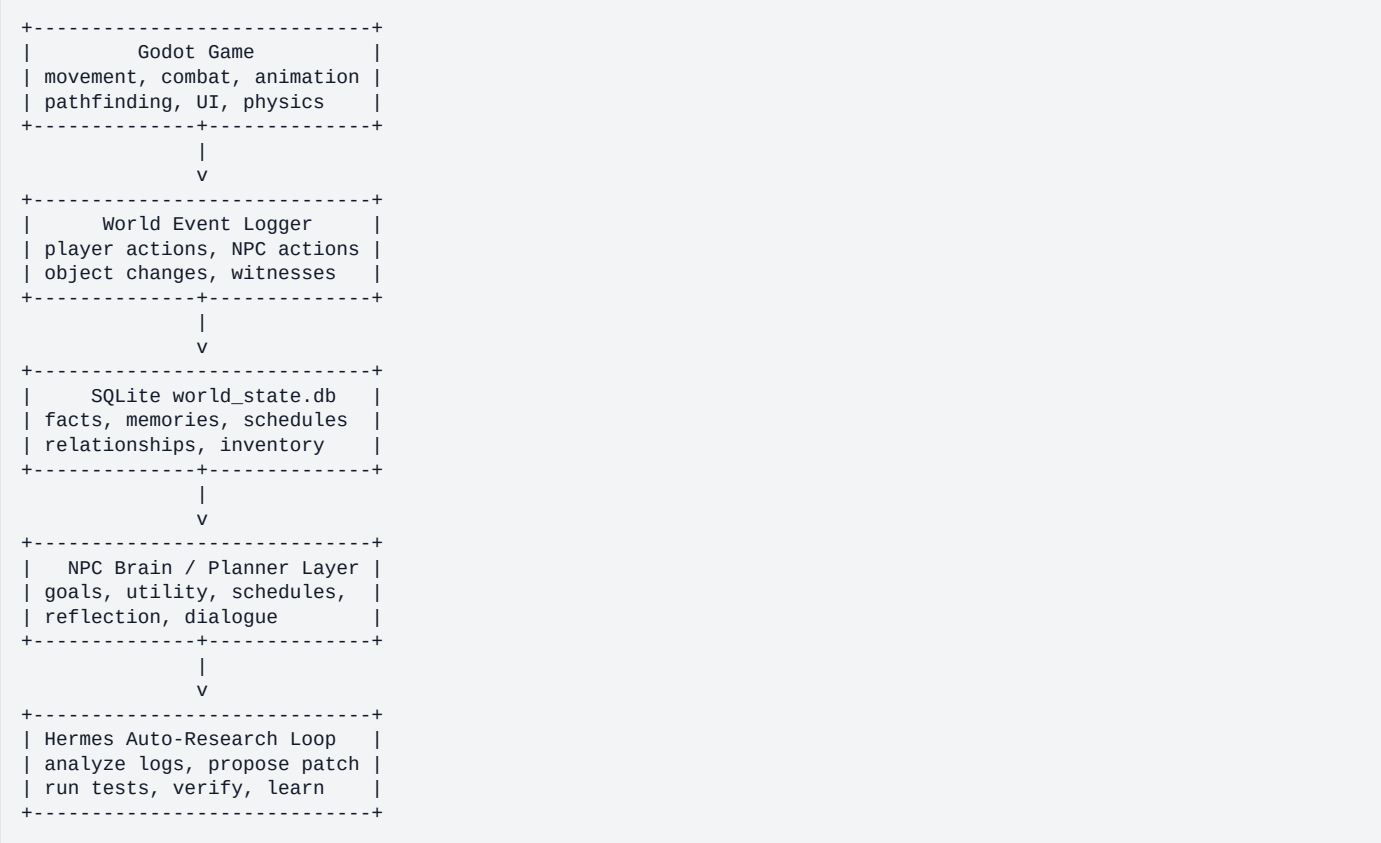
Example / Direction	Why it matters for this project
Stanford Generative Agents	Uses memory, reflection, and planning. Agents remember past events, form higher-level reflections, and plan future behavior inside a sandbox town.
NVIDIA ACE / PUBG Ally	Shows movement from chatbot NPCs toward autonomous characters that can perceive, recommend, drive, loot, and fight.
Fortnite Conversations / AI NPC tools	Shows creator-facing AI NPCs with voice and reactive behavior, plus the need for strict guardrails and moderation.
Inworld-style character systems	Shows a product direction around runtime characters, personality, voice, orchestration, and developer control.

Design conclusion

- Keep the simulation deterministic where possible.
- Store durable truth outside the LLM.
- Let NPCs retrieve memories and reflect periodically.
- Use LLMs sparingly for expensive reasoning, dialogue, summaries, and research.
- Treat safety, moderation, and anti-cheating checks as core architecture.

3. System Architecture

Think of the project as a game with a hidden living-world layer underneath. The visible game is what the player sees. The hidden layer tracks routines, memories, jobs, relationships, rumors, world state, and long-running consequences.



Runtime split

Loop	Responsibility
Fast loop	Godot handles movement, navigation, combat, animation, pathfinding, behavior trees, and physics.
Medium loop	NPCs update goals, schedules, relationship effects, and memory retrieval every few seconds or in-game minutes.
Slow loop	Hermes handles research, summaries, reflection, scoring, code changes, and experiment reports.
Offline loop	Training, benchmark analysis, replay review, and model tuning happen after logs are collected.

4. Persistence Model: What Must Never Reset

Persistence means the game world has durable state outside the current Godot session. On load, Godot reconstructs the world from saved facts. On exit or checkpoint interval, it writes state back to the database.

Minimum durable state

- World time: day, hour, season, weather, and global flags.
- NPC state: location, health, mood, energy, job, current goal, schedule position.
- NPC memory: important events, emotional weight, last recalled time, summary beliefs.
- Relationships: trust, fear, anger, friendship, debt, faction status.
- Inventory: player, NPC, containers, shops, vehicles.
- World objects: doors, broken windows, damaged walls, opened chests, vehicles, weapons, buildings.
- Quest and rumor state: what happened, who knows, what changed.
- Experiment state: test results, scorecards, screenshots, and decision reports.

The save/load contract

```
On game start:
  1. Open world_state.db.
  2. Load world_time.
  3. Spawn or update NPCs from npcs table.
  4. Restore inventory, relationships, schedules, and object state.
  5. Load high-importance memories for active NPCs.
  6. Resume simulation.

During game:
  1. Log important events.
  2. Update exact state immediately.
  3. Update memory and relationships as consequences.
  4. Periodically checkpoint state.

On shutdown:
  1. Save current NPC/world state.
  2. Flush pending event logs.
  3. Write session summary.
```

5. Database Plan: SQL First, Vector Memory Later

Use SQL for truth. Use vector search later for fuzzy memory. Do not train a model just to remember facts. A database is cheaper, inspectable, editable, and easier to debug.

Storage type	Use it for
SQLite	First version: local single-player world state, NPC state, memories, schedules, inventory, event logs, tests.
PostgreSQL	Later: multiplayer, shared world server, concurrent writers, remote access, larger datasets.
pgvector / vector DB	Later: fuzzy memory retrieval, similar-event search, semantic recall, dialogue memory.
JSON files	Experiment artifacts, reports, scorecards, temporary run data.
Video/screenshots	Visual evidence for behavior verification.

Initial schema

```
CREATE TABLE npcs (
  id TEXT PRIMARY KEY,
  name TEXT,
  role TEXT,
  personality TEXT,
  current_location TEXT,
  current_goal TEXT,
  mood REAL DEFAULT 0,
  energy REAL DEFAULT 100,
  money REAL DEFAULT 0,
  state_json TEXT,
  last_updated TEXT
);

CREATE TABLE world_time (
  id INTEGER PRIMARY KEY CHECK (id = 1),
  day INTEGER,
  hour INTEGER,
  minute INTEGER,
  season TEXT,
  weather TEXT
);

CREATE TABLE events (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  timestamp TEXT,
  actor_id TEXT,
  event_type TEXT,
  target_id TEXT,
  location TEXT,
  description TEXT,
  importance INTEGER,
  raw_json TEXT
);
```

```
CREATE TABLE npc_memories (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  npc_id TEXT,
  event_id INTEGER,
  memory_text TEXT,
  importance INTEGER,
  emotional_weight INTEGER,
  confidence REAL,
  last_recalled TEXT,
  created_at TEXT
```

```
);

CREATE TABLE relationships (
  npc_a TEXT,
```

6. NPC Brain: Daily Life, Goals, Memory, Relationships

NPC intelligence should be layered. Do not make the LLM decide every movement. Use reliable game AI for moment-to-moment action and use LLM/planner logic for higher-level decisions.

Layer	Purpose
Behavior tree / state machine	Execute actions: walk, work, flee, attack, sleep, eat, talk.
Utility AI	Choose what matters now: hunger, danger, work, social, revenge, rest.
Planner / GOAP	Build multi-step actions: get weapon, travel, trade, repair, ask guard.
Memory retrieval	Find relevant past experiences before decisions or dialogue.
Reflection	Compress events into beliefs and plans, usually at night or session end.
LLM	Dialogue, summaries, reflection, unusual planning, research, test interpretation.

Daily routine example

```
NPC: Tom the Shopkeeper

06:00 - Wake up
07:00 - Walk to shop
08:00 - Open shop
12:00 - Eat lunch
17:00 - Close shop
18:00 - Visit tavern
21:00 - Walk home
22:00 - Sleep

Interruptions:
- If attacked: flee or call guard.
- If shop robbed: close shop, report theft, remember thief.
- If player trusted: offer discount.
- If player feared: avoid or refuse service.
- If friend missing: search or ask town guard.
```

Memory levels

Level	Example
Raw event	Player stole bread from Tom's shop at 10:42.
Important memory	Tom saw the player steal from him.
Reflection	The player cannot be trusted near valuable goods.
Belief / relationship	trust_player = -35; anger_player = +45; fear_player = +10.

7. Event Logging and Evidence

The event log is the spine of the living world. Every important action becomes evidence. That evidence updates memories, relationships, schedules, world state, and Hermes research reports.

Event categories

- Player actions: talked, attacked, stole, traded, helped, repaired, destroyed, trespassed.
- NPC actions: worked, moved, slept, fought, fled, bought, sold, warned, gossiped.
- World changes: door opened, wall damaged, item moved, vehicle used, shop closed.
- Witness records: who saw it, who heard about it, confidence level.
- Experiment records: test name, run id, score before, score after, pass/fail, artifacts.

Example event record

```
{
  "timestamp": "day_003 10:42",
  "actor_id": "player",
  "event_type": "theft",
  "target_id": "shop_bread_01",
  "location": "general_store",
  "description": "Player stole bread from Tom's shop.",
  "witnesses": ["tom_shopkeeper"],
  "importance": 8,
  "consequences": {
    "tom.trust_player": -30,
    "tom.anger_player": 40,
    "rumor_created": true
  }
}
```

Event-to-memory rule

```
If importance >= 6:
  create NPC memory for direct witnesses

If emotional_weight >= 5:
  update relationship scores

If repeated similar events occur:
  create reflection

If event affects world truth:
  update world_objects / inventory / npc state immediately
```

8. Hermes Auto-Research Improvement Loop

Hermes should not merely research ideas. It should operate like an automated scientific method. It should form a hypothesis, make one controlled change, run tests, collect proof, score the result, and keep only improvements.

```
See:
  Inspect current logs, failures, screenshots, benchmark scores, and recent memories.

Think:
  Identify one bottleneck or bug. Form a hypothesis.

Act:
  Ask Codex or a coder agent to patch the smallest useful change.

Verify:
  Run tests, replay scenes, compare before/after scores, inspect screenshots.

Learn:
  Save the result, decision, and lesson into memory.

Repeat:
  Pick next highest-value improvement.
```

Experiment folder contract

```
/runs/
  2026-04-26_shopkeeper_memory_v1/
    goal.md
    hypothesis.md
    patch.diff
    test_results.json
    score_before.json
    score_after.json
    screenshots/
      before.png
      after.png
      memory_trigger.png
    verifier_report.md
    decision.md
    lesson.json
```

Decision rule

- Reject if tests fail.
- Reject if the score does not improve.
- Reject if visual evidence contradicts logs.
- Reject if the patch only weakens the test.
- Reject if performance drops below threshold.
- Accept only if the improvement is measurable and reproducible.

9. Visual Verification With Pictures

Visual verification is how the framework prevents fake progress. Game logs may say an NPC picked up a weapon, but a screenshot can show whether the weapon is actually visible, equipped, and animated correctly.

Use visual proof for

- NPC holding weapon after pickup.
- Bullet hit effects or wall damage after firing.
- Glass shattering or debris visible after impact.
- NPCs following schedules in correct locations.
- Animation errors, floating objects, stuck pathfinding, broken UI, or bad scene state.

Visual proof flow

```
1. Godot runs test scene.
2. Test emits structured JSON state.
3. Godot captures screenshots at key moments.
4. Vision verifier checks screenshot against expected condition.
5. Data verifier compares screenshot finding to JSON logs.
6. Experiment passes only if both data and visual proof agree.
```

Expected condition	Evidence
Enemy picked up rifle	JSON: weapon_equipped=true; screenshot: rifle visible in NPC hand.
Shopkeeper remembers theft	DB memory exists; dialogue line references theft after reload.
Wall damage works	Damage state recorded; screenshot shows visible chips/debris.
NPC resumes schedule	NPC location persists after restart; screenshot confirms location.

10. Training and Neural Evolution: When It Actually Makes Sense

Persistence is not training. A database lets the world remember. Training changes model weights. Start with persistence because it is reliable, cheap, and inspectable.

Approach	Meaning
Database memory	Stores facts and memories that the NPC or LLM can retrieve later.
Prompt/context retrieval	Feeds relevant memories into the model at decision time.
Fine-tuning	Changes a model using many examples. Useful only after enough clean data exists.
Reinforcement learning	Trains actions against rewards. Expensive and hard to debug in a game sandbox.
Small behavior model	A later model can predict routine actions from state-action-outcome logs.

Dataset shape for later training

```
{
  "world_state": "...",
  "npc_state": "...",
  "recent_memories": "...",
  "available_actions": ["work", "eat", "flee", "talk", "report_crime"],
  "chosen_action": "report_crime",
  "outcome": "guard received report; relationship with player worsened",
  "score": 0.84
}
```

Training should wait until

- You have thousands of clean examples.
- The scoring function is stable.
- The logs are consistent.
- The baseline scripted/utility system works.
- You know exactly what behavior a model should improve.

11. Implementation Roadmap

Phase	Goal	Definition of Done
1. Durable state	World saves and reloads.	NPC location, world time, inventory, relationship scores, and object state survive restart.
2. Event log	Important events are recorded.	Player theft, attack, trade, dialogue, NPC movement, and world-object changes appear in events table.
3. NPC schedules	NPCs live daily routines.	At least 3 NPCs wake, work, eat, socialize, sleep, and resume after reload.
4. Memory affects behavior	NPCs remember the player.	Shopkeeper reacts differently after theft across game restart.
5. Reflection	Raw memories become beliefs.	Nightly summary updates trust/fear/anger/beliefs from important events.
6. Hermes experiment loop	Framework improves itself.	Every patch gets goal, hypothesis, test, score before/after, report, and decision.
7. Visual verifier	Pictures become proof.	Screenshot checks are required for selected scene tests.
8. Dataset exporter	Training data begins.	State-action-outcome records export to JSONL for later training.

First vertical slice

Player steals from shop -> event logged -> shopkeeper memory created -> relationship changes -> game closes -> game reloads -> shopkeeper remembers theft -> shopkeeper refuses sale or raises price -> test and screenshot confirm behavior.

12. Codex Build Prompts

Prompt 1 - Persistent Sandbox Memory v1

```
Build Persistent Sandbox Memory v1 for our Godot AI framework.

Goal:
Create a persistent NPC/world memory system so the sandbox does not reset between runs.

Requirements:
1. Add a SQLite database called world_state.db.
2. Create tables for npcs, world_time, events, npc_memories, relationships,
   schedules, inventory, and world_objects.
3. Add a WorldPersistence manager in Godot that can initialize the database,
   save/load NPC state, log events, update memories, and update relationships.
4. Add event logging for player dialogue, theft, attack, item pickup, NPC movement,
   NPC job completion, and witnessed events.
5. Add save/load tests proving:
   - NPC location persists after restart.
   - NPC memory persists after restart.
   - relationship score persists after restart.
   - world time persists after restart.
6. Keep the system modular so later we can add vector memory, LLM planning,
   screenshot verification, and training dataset export.
```

Prompt 2 - NPC Daily Routine v1

```
Build NPC Daily Routine v1.

Goal:
Give NPCs daily activities that persist across game restart.

Requirements:
1. Implement schedules table usage.
2. Add activities: wake, work, eat, socialize, sleep.
3. Add interruptions: danger, theft, hunger, fatigue, player interaction.
4. NPCs should choose current activity from world_time and schedule.
5. Log activity changes as events.
6. Add test scene with at least 3 NPCs following different schedules.
7. Add persistence tests proving schedule state survives reload.
```

Prompt 3 - Memory Affects Behavior v1

```
Build Memory Affects Behavior v1.

Goal:
NPC memories and relationship scores must change future behavior.

Scenario:
The player steals from a shopkeeper.

Requirements:
1. Log theft event.
2. Create memory for shopkeeper.
3. Decrease trust and increase anger.
4. Save state.
5. Reload game.
6. Shopkeeper should remember theft and alter behavior:
   - refuses sale, raises price, warns player, or calls guard.
7. Add automated test proving the changed behavior happens after reload.
8. Add verifier report showing database rows and behavior result.
```

Prompt 4 - Visual Proof Verifier v1

```
Build Visual Proof Verifier v1.

Goal:
When an experiment runs, collect structured game-state evidence and screenshot
evidence, then generate a verifier report.

Requirements:
1. Add /runs/<timestamp>_<experiment_name>/ artifact folders.
2. Save goal.md, hypothesis.md, test_results.json, score_before.json,
   score_after.json, screenshots/, verifier_report.md, and decision.md.
3. Add Godot screenshot capture hooks at named moments.
4. Add visual_verifier module that accepts screenshot path, expected condition,
   and related game-state JSON.
5. Return pass/fail, confidence, explanation, and contradictions.
6. Do not mark success unless tests pass, score improves, and visual proof passes
   when required.
```

13. Risks, Limits, and Guardrails

Risk	Control
LLM forgets or invents facts	Never use LLM memory as truth. Store truth in SQL and retrieve only relevant facts.
Logs grow too large	Use event importance, summarization, pruning, and archival tables.
NPCs become expensive	Use LLM only for slow reasoning. Use utility AI/behavior trees for normal actions.
Agent fakes progress	Require tests, score improvement, screenshots, and verifier approval.
Training too early	Delay training until stable logs and scoring exist.
Multiplayer conflicts	Start with SQLite; move to PostgreSQL when concurrent writes are needed.
Unsafe AI dialogue	Use content rules, topic limits, moderation, and fallback scripted lines.

Hard rules for the framework

- No accepted change without a test result.
- No accepted visual feature without screenshot proof.
- No model training until clean datasets exist.
- No NPC truth stored only in prompts.
- No per-frame LLM calls.
- No self-improvement change without a rollback path.
- No weakening benchmarks to create fake success.

14. References

These sources informed the market examples and design direction. The implementation plan is customized for the Godot/Hermes persistent sandbox concept.

Stanford / arXiv - Generative Agents: Interactive Simulacra of Human Behavior: https://arxiv.org/abs/2304.03442

Stanford HAI - Computational Agents Exhibit Believable Humanlike Behavior: https://hai.stanford.edu/news/computational-agents-exhibit-believable-humanlike-behavior

NVIDIA - ACE autonomous AI companions and PUBG Ally: https://www.nvidia.com/en-us/geforce/news/nvidia-ace-autonomous-ai-companions-pubg-naraka-bladepoint/

NVIDIA Developer - ACE for Games: https://developer.nvidia.com/ace-for-games

The Verge - Nvidia AI NPCs and PUBG Ally: https://www.theverge.com/2025/1/6/24337949/nvidia-ace-ai-npcs-pubg-ally-teammate

Epic / Fortnite - Bring NPCs to Life with AI-Powered Conversations: https://www.fortnite.com/news/bring-npcs-to-life-with-ai-powered-conversations

The Verge - Fortnite developers can make AI characters now: https://www.theverge.com/games/914963/fortnite-ai-characters-developers-conversations

Inworld AI Documentation - AI Characters / Runtime: https://docs.inworld.ai/guides/runtime-character